

Introduction to Digital Systems and Number Systems

2.1.1 Need for Digital Systems

A system is a group of elements or components that are integrated to achieve a specific goal. A signal is a way of conveying some information. It refers to any time-varying voltage, current, or electromagnetic wave that carries information. Information could be in the form of an image, sound, temperature, etc. There are two types of signals: analog signals and digital signals. Analog signals are continuous signals, and they vary with respect to time. Digital signals are discrete signals that are present only at discrete instants of time.

Based on the type of signal that the system handles, there are two types of systems: analog and digital. Analog systems process analog signals; for example, volume control on old radios, telephone handsets, and TR-10 desktop analog computers. Since analog signals involve continuous signals, it requires huge memory to store the collected information. Also, designed systems cannot be easily modified in real-time.

Digital systems process digital signals in the form of discrete signals, which are obtained by sampling analog signals. The process of converting analog signal to digital signal is shown in Figure 1. A signal sampler is a kind of high-speed switch that takes an analog signal as an input and produces discrete signals at predefined time intervals. Such signals have a finite set of possible amplitudes. The best example of a digital system is a computer that uses digital signals in the form of square waves. This kind of digital signal has only two amplitudes or voltage levels, namely high voltage level and low voltage level.

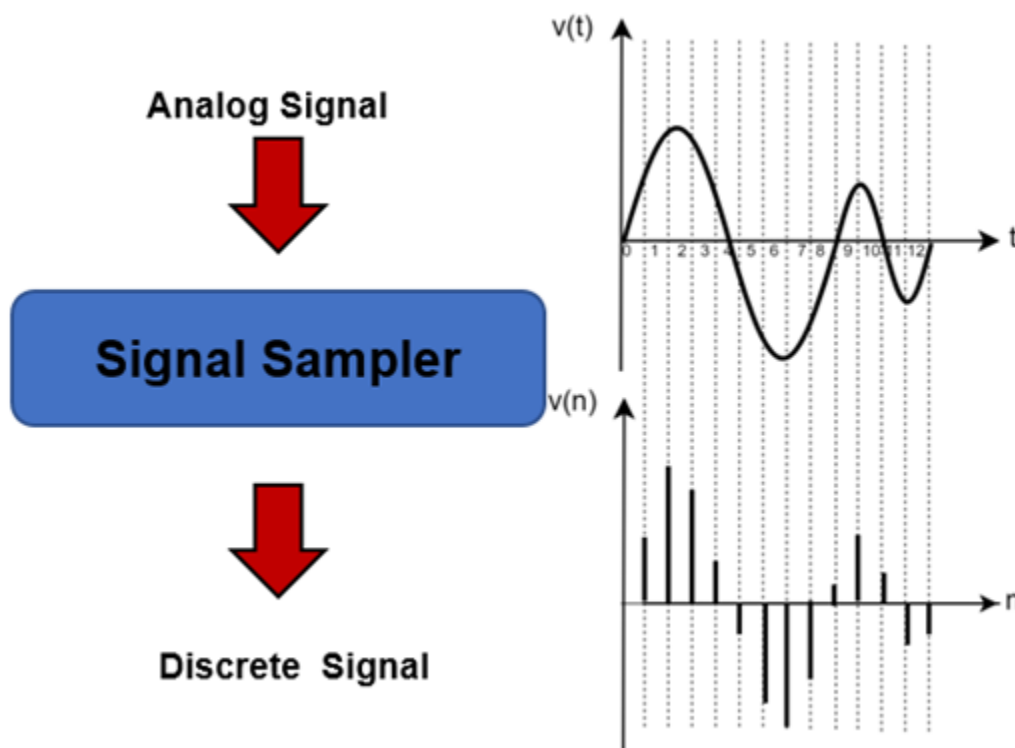


Figure 1: Analog to digital signal (discrete signal) conversion.

Digital systems are popular because of the advantages it offers, some of which are given as follows:

- Easy to design and implement
- Cost-effective
- More flexible design
- Easy data storage, compression, and encryption
- More reliable—the impact of noise interference and distortion is very less

Since digital signals are obtained by sampling analog signals, the accuracy of the digital signal obtained depends on the sampling rate. If the sampling rate is low, sampled information will be irreversibly lost, and the original signal will not be represented correctly. A higher sampling rate provides better accuracy but will have an impact on storage capability and the amount of time it takes to process the information. There is always a compromise between sampling rate, processing time, and storage.

Since digital systems are more reliable, they are employed in almost all applications, including computer systems.

2.1.2 Introduction to Number System

A number system defines how a number can be expressed. These numbers are represented using distinct digits and or symbols in a consistent manner. In general, the value of any digit in a number can be determined by:

- The digit or a symbol
- Its position in the number
- The base or radix value of the number system

The classification of the number system is shown in Figure 2. Non-positional number system uses symbols to represent any number. Hence, it is also known as the “Symbolic” number system.

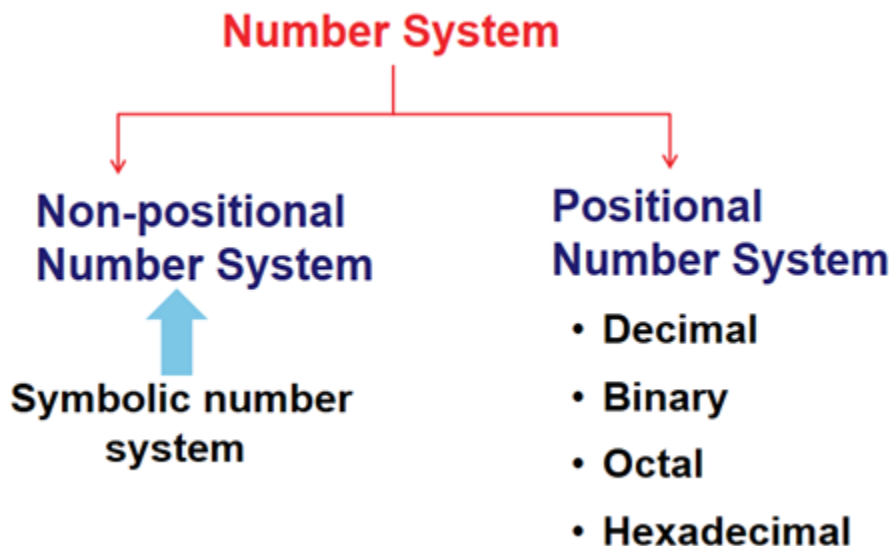


Figure 2: Classification of number system.

For example, the Roman number system uses only seven symbols I, V, X, L, C, D, and M. In this number system, a set of rules are followed to represent a number. Figure 3 shows the Roman number system symbols and corresponding values in decimal.

Symbols	I	V	X	L	C	D	M
Value in decimal	1	5	10	50	100	500	1000

Figure 3: Roman number system.

As the name indicates, in a positional number system, the position of a symbol determines the value it represents. Examples of number systems include decimal, binary, octal, and hexadecimal.

Figure 4 shows the general representation of a number in a positional number system. In general, any number $N(b)$ in the positional number system is divided into two parts: the integer part and the fractional part. These two parts are separated by a radix or a decimal point.

$$N(b) = d_{n-1} d_{n-2} \dots d_1 d_0 . d_{-1} d_{-2} \dots d_{-m}$$

Figure 4: General representation of a number in the positional number system.

In the expression given in Figure 4, N represents the number, b represents the radix or base of a number, small n represents the number of digits in the integer portion, and m is the number of digits in the fractional portion. The commonly used number system is shown in Figure 5.

Number System	Base or Radix	Symbols Used	Example
Binary	2	0, 1	11101.11
Octal	8	0,1,2,3,4,5,6,7	234.70
Decimal	10	0,1,2,3,4,5,6,7,8,9	92812.15
Hexadecimal	16	0,1,2,3,4,5,6,7,8,9, A,B,C,D,E,F	A123.FE

Figure 5: Example number systems.

In binary, bi means 2. The binary number system uses only two symbols or digits 0 and 1. Since there are two distinct symbols, the base of the binary number system is 2. In the octal number system, oct means 8. Base of the octal number system is 8 and uses eight symbols 0 to 7. In the decimal number system, deci means 10. The base of the decimal number system is 10 and uses ten symbols from 0 to 9. Hex in a hexadecimal number system means 6, and deci means 10. The hexadecimal number system uses a base value of 16 and uses 16 distinct symbols to represent a number. It uses 0 to 9 and A to F symbols.

Decimal Number System

The decimal number system has a base value of 10 and uses ten symbols from 0 to 9. Consider 1248.84 as an example. As shown in Figure 6, this number has two parts, that is, the integer part and the fractional part, and they are separated by a decimal point.

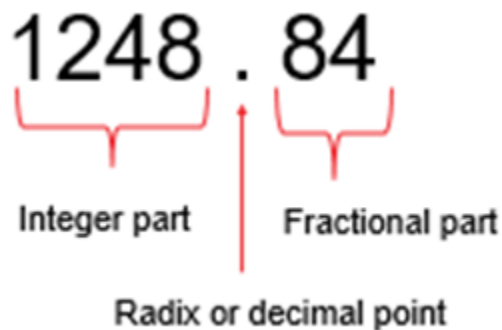


Figure 6: Example of decimal number representation.

In the decimal number system, number representation is algorithmic. Every digit has a position number. The integer part has a positive position number, whereas the fractional part has a negative position number. Each digit is associated with a weight and is equal to 10 power position numbers. Figure 7 shows the position number and weights associated with the decimal number 1248.84. The final number is obtained by adding all the terms obtained by digits multiplied by their respective weights.

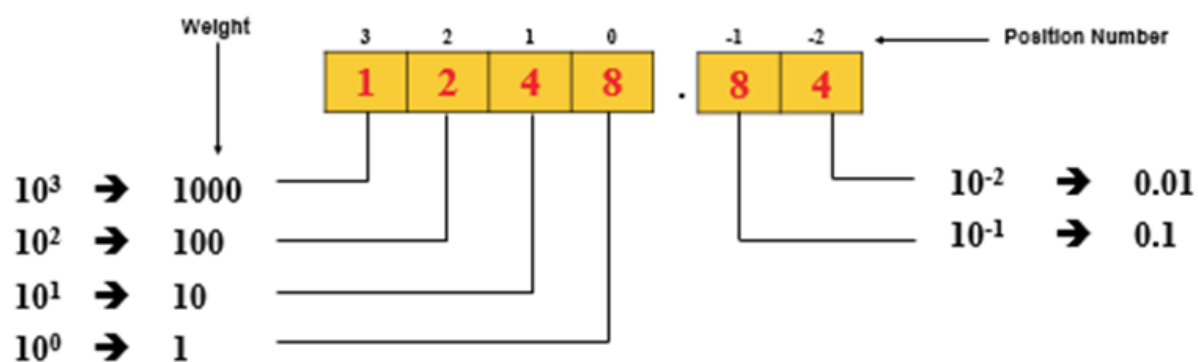


Figure 7: Position number and weights associated with decimal number 1248.84.

The main advantages of the decimal number system are that they are easy to represent, read, and manipulate. Hence, it is most used in daily life. However, decimal number representation cannot be employed in computers for the following reasons:

- **Hard to store:**

This is because each digit in a number is made of ten symbols. The first electronic computer ENIAC used ten vacuum tubes per digit, and the computer design was very bulky.

- **Hard to transmit:**

Encoding ten different symbols requires ten signal levels so that it can be transmitted over a single wire. Also, to minimize the errors, high-precision encoding is needed.

- **Complex design and implementation:**

Design requires handling ten signal levels. Hence, it complicates the implementation of various arithmetic and logic functions, such as addition and multiplication.

2.1.3 Binary Number System

The binary number system uses only two possible values, 0 or 1, for each digit in a number. This number system is used in every digital system because of easy storage, easy transmission, and easy implementation of required functions. In such systems, everything is represented by either the presence or absence of a signal. The presence of a signal is usually interpreted as digit 1, and the absence of a signal as digit 0.

Each digit in a binary number is known as a bit. The binary number is made up of several bits. The rightmost bit is known as the least significant bit, LSB, whereas the leftmost bit is known as the most significant bit, MSB. In a digital system, MSB is the bit that has the largest weight in a binary number, whereas LSB has the least weight. Figure 8 shows the position of MSB and LSB for two example binary numbers.

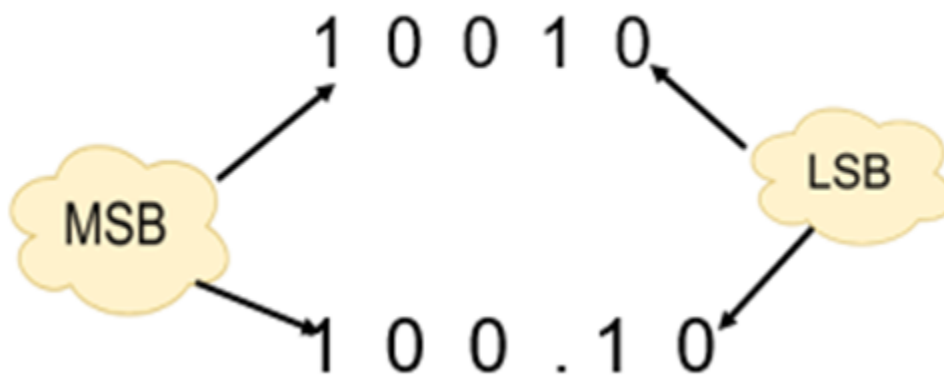


Figure 8: Position of MSB and LSB.

Since a binary number system is a positional number system, each bit has a positional value and weight associated with it. Figure 9 shows the example binary number 11010.11, which has integer and fractional parts. Before the decimal point, that is the integer part, the bits are represented as b_0 , b_1 , b_2 , b_3 , and b_4 , and the corresponding weight is 2^0 , 2^1 , 2^2 , 2^3 and 2^4 . After the decimal point, bits are represented as b_{-1} , b_{-2} , etc.; and their corresponding weight is 2^{-1} , 2^{-2} , etc.

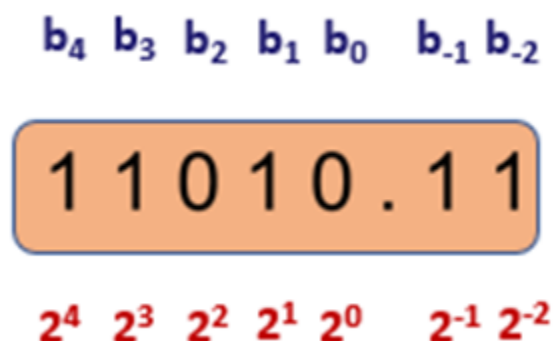


Figure 9: Position value and weight associated with the position.

Usually, programming languages use a standard number of bits while organizing data storage and access.

- A **Nibble** □ 4 bits
- A **Byte** □ 8 bits
- A **Word** □ 16 bits or 2 bytes

- A **D-word** □ 32 bits or 4 bytes
- A **Q-word** □ 64 bits or 8 bytes

Decimal to Binary Conversion

While converting from decimal to binary, integer and fractional parts are converted separately. For converting the integer part to binary, the successive division method is used, whereas for converting the fractional part, successive multiplication is used. For both division and multiplication, the target number system's base, which is base 2, is considered. In general, while converting the integer part, first divide the integer part of the decimal number by 2. The first remainder that is obtained will be the LSB of the binary number. If the quotient is zero, the conversion is complete. If the quotient is not zero, then the quotient will be divided by 2. The new remainder is the next most significant bit of the binary number. This continues until the quotient becomes 0.

For fractional part conversion, multiply the fractional part by 2. The carry generated is kept aside, and the fraction will then be multiplied by 2. This process is continued until the fractional part is zero or the required accuracy is attained. Figure 10 shows the decimal number 125.66 to binary conversion.

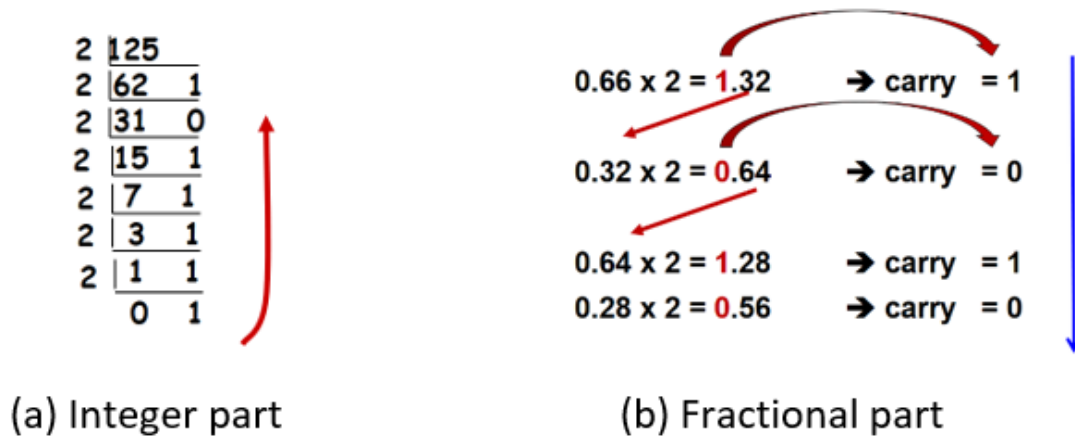


Figure 10: Decimal 125.66 to binary conversion.

Binary to Decimal Conversion

To convert binary to decimal, the following steps are followed.

Step 1: Multiply each bit of the binary number by its corresponding bit-weighting factor.

Step 2: Sum up all the products to get the decimal number.

Conversion process for a binary number 1010.101 is shown in Figure 11.

2^3 2^2 2^1 2^0 2^{-1} 2^{-2} 2^{-3}
 1 0 1 0 . 1 0 1

$$1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}$$

$$1 \times 8 + 0 \times 4 + 1 \times 2 + 0 \times 1 + 1 \times \frac{1}{2} + 0 \times \frac{1}{4} + 1 \times \frac{1}{8}$$

10.625

Figure 11: Binary to decimal conversion.

Bit Permutation

In general, in any number system, the total numbers that are represented depend on the number of digits in that number. In a binary number system, if a number sequence has n bits, then we can have 2^n possible combinations or permutations; for example, if $n = 1$, then there are two combinations, that is, 0 and 1. Similarly, when $n = 2$, there will be four combinations, 00, 01, 10, and 11. Figure 12 shows the permutation table for $n = 1, 2$, and 3.

$n = 1$ bit

Binary	Decimal
0	0
1	1

$n = 2$ bits

Binary	Decimal
0 0	0
0 1	1
1 0	2
1 1	3

$n = 3$ bits

Binary	Decimal
0 0 0	0
0 0 1	1
0 1 0	2
0 1 1	3
1 0 0	4
1 0 1	5
1 1 0	6
1 1 1	7

Figure 12: Permutation table for $n = 1, 2$, and 3.

2.1.4 Octal Number and Hexadecimal Number System

Octal number system is an example of a positional number system that uses eight symbols 0 to 7. The base of the octal number system is 8. The conversion process is similar to binary, which is shown in Figure 13. To convert decimal numbers to octal numbers, the integer part and fractional part are treated separately. The integer part is divided by eight until the quotient becomes 0. The remainder that is generated by the process of division will give a decimal equivalent of the octal integer part. To convert the fractional part to decimal, successive multiplication is used. To convert from an octal number to a decimal, weighted multiplication is used.

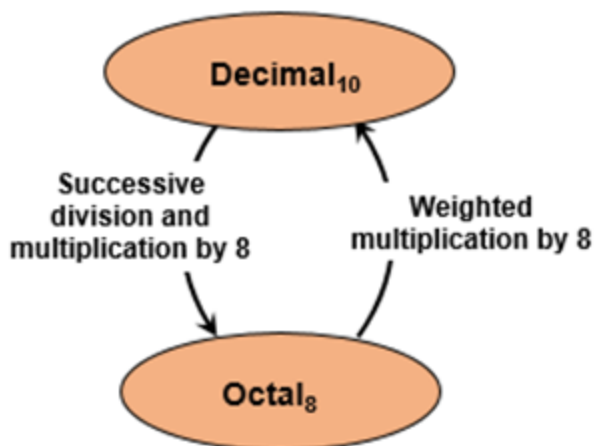


Figure 13: Conversion between decimal and octal number systems.

Figure 14 shows the example where the decimal number 156.32₍₁₀₎ is converted to an octal number. To convert decimal to octal, weighted multiplication is used. Figure 15 shows the conversion of 761.5₍₈₎ to a decimal number.

$$156.32_{(10)} = 432.2436_{(8)}$$

$ \begin{array}{r} 8 \overline{) 156} \\ 8 \overline{) 19} \quad 4 \\ 8 \overline{) 2} \quad 3 \\ 0 \quad 2 \end{array} $	$ \begin{aligned} 0.32 \times 8 &= 2.56 \rightarrow 2 \\ 0.56 \times 8 &= 4.48 \rightarrow 4 \\ 0.48 \times 8 &= 3.84 \rightarrow 3 \\ 0.84 \times 8 &= 6.72 \rightarrow 6 \end{aligned} $
--	---

Figure 14: Conversion of 156.32₍₁₀₎ to octal number.

$$\begin{array}{ccccccc}
 8^2 & & 8^1 & & 8^0 & & 8^{-1} \\
 7 & & 6 & & 1 & . & 5 \\
 \\
 & = & 7 \times 8^2 & + & 6 \times 8^1 & + & 1 \times 8^0 & + & 5 \times 8^{-1} \\
 & = & 497.625_{(10)}
 \end{array}$$

Figure 15: Conversion of 761.5₍₈₎ to decimal number.

Binary to Octal Conversion

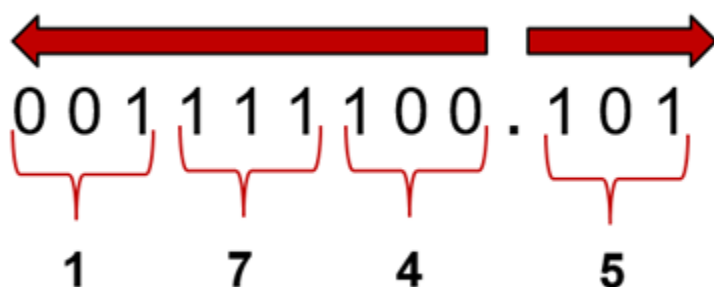
There are two methods to convert a binary number to an octal number.

- **Method 1:** First, convert the binary number to decimal number and then to octal number.

Example: $(1111100.101)_2 \square (124.625)_{10} \square (174.5)_8$

- **Method 2:** Make 3 bits group.

Figure 16 shows an example of how a binary number 1111100.101 is converted using Method 2. The direction of grouping for the integer part and fractional part is shown. For integers, part bits are grouped from right to left, and for the fractional part, bits are grouped from left to right. If the last group has less than three bits, then pad it with zeros and convert.



Octal	Binary
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Figure 16: Binary to octal conversion.

Octal to Binary:

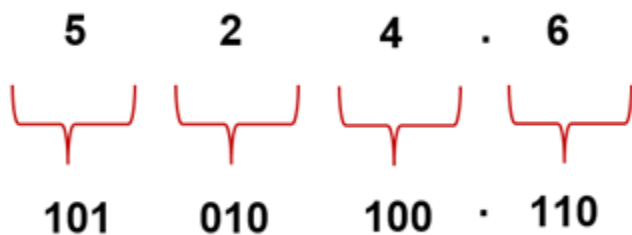
There are two methods to convert an octal number to binary.

- **Method 1:** Convert octal to decimal and then to binary.

Example: $(524.6)_8 \square (340.75)_{10} \square (101010100.110)_2$

- **Method 2:** Convert individual octal digits to binary.

Figure 17 shows an example of how a binary number 1111100.101 is converted using Method 2. As shown in the figure, individual octal digits are converted to binary.



Octal	Binary
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Figure 17: Octal to binary conversion.

Hexadecimal Number System

Hexadecimal number system uses 16 symbols: 0 to 9 and A to F. Software developers widely use hexadecimal numbers as they provide a human-friendly representation of binary-coded values. It is

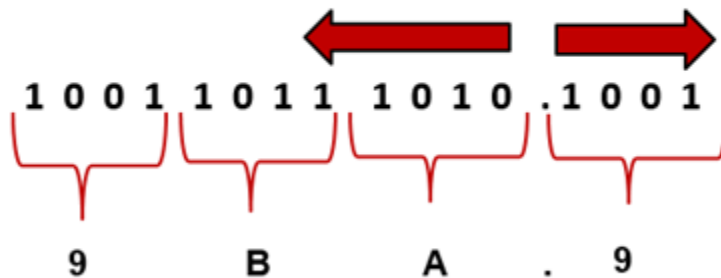
shorter and easier to read than binary and octal. Each hexadecimal digit represents one nibble, which is four binary bits.

Binary to Hexadecimal Conversion

There are two ways of converting binary to hexadecimal.

- **Method 1:** Convert from binary to decimal and then decimal to hexadecimal.
- **Method 2:** Group four bits and then convert those four bits to equivalent hexadecimal digits.

Figure 18 shows the conversion of a binary number to a hexadecimal number using Method 2. For the binary number integer part, the four-bit group is formed from right to left, whereas for the fractional part, four-bit groups are formed from left to right.



Hexadecimal	Binary
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111

Figure 18: Binary to hexadecimal number conversion.

Hexadecimal to Binary Conversion

To convert from hexadecimal to binary, individual hexadecimal digits are converted to binary. Figure 19 shows the conversion of a hexadecimal number to a decimal number.

Hexadecimal	Binary
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111

4	A	B	.	8
0100	1010	1011	·	1000

Figure 19: Hexadecimal to binary number conversion.

2.1.5 Operations on Binary

Bitwise Addition

A logic circuit is an essential building block of a logic system that will have several inputs and outputs. In any logic system design, the truth table plays an important role. The truth table lists all possible combinations of input binary variables and the corresponding outputs of a logic system. When the number of input binary variables is one, there are only two possible input combinations: "0" and "1." If the number of inputs is two, there can be four possible input combinations, that is, 00, 01, 10, and 11. Similarly, the number of possible input combinations for three input binary variables becomes eight. So, if a logic circuit has n binary inputs, its truth table will have 2^n possible input combinations.

Consider two inputs A and B. There are two outputs, Sum and Carry. Figure 20 shows the truth table consisting of inputs and outputs. Since there are two inputs, there are four possible combinations.

- When A = 0 and B = 0, adding 0 with 0 yields the sum as zero, with no carry, which is indicated as zero in the truth table.
- When A = 0 and B = 1, adding 0 with 1 yields the sum as 1, with no carry.
- When A = 1 and B = 0, adding 1 with 0 yields the sum as 1, carry as 0.
- When A = 1 and B = 1, adding 1 with 1 will result in 0 and carry 1.

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Figure 20: Truth table for the addition of two bits.

1's Complement

Complementation is one of the very important operations on bits. There are two types of complements: 1's complement and 2's complement.

In 1's complement operation, we flip or invert all the bits in a number; that is, change 0 to 1 and 1 to 0. For example, 1's complement of 1100 is 0011.

2's Complement

2's complement is 1's complement + 1. For example, 2's complement of binary number 011010000 is computed as follows:

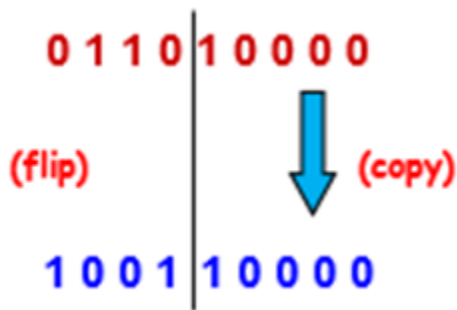
Original number → 011010000

1's complement → 100101111

Plus 1 → _____1

2's complement → 100110000

There is a shortcut method. Copy bits from right to left until (and including) the first 1, and then flip all the remaining bits. Figure 21 shows the shortcut method.



Introduction to K-Map

4.1.1 Introduction to K-Map Method

Complex logic expressions can be simplified using well-known Boolean algebra, which is composed of several useful Boolean Postulates and theorems. But this method comes with two main drawbacks.

- Complex Boolean expressions require several time-consuming steps for simplification. There are no specific rules to guide us on which theorems should be applied and when. Also, when a theorem or a postulate is applied, it is difficult to determine whether the simplest form has been achieved or not.
- This technique is error-prone. There are chances that one may end up making mistakes.

Another technique, K – map or Karnaugh map, is an easy and systematic method of simplifying Boolean Expressions. It is a graphical method developed by Maurice Karnaugh in 1953 at Bell Labs [Figure 1]. Using this map technique, one can achieve the most simplified expression either in the Sum of Products, SOP form, or products of sums, POS form. The simplified expression will have a minimum number of terms with the least number of literals in each term.



Figure 1: Maurice Karnaugh.

A K-map is a collection of cells or squares. Each cell represents a minterm or a maxterm. The group of these cells represents the output values of a given Boolean function. These cells are arranged such that the adjacent cells differ in the value of one variable. This is a condition for cell adjacency. An n – variable K-map will have 2^n cells. For example, a 2-variable K map has four cells. A 3-variable K map has 23, that is, eight cells. Similarly, the 4 variable K map has 16 cells and so on. This technique can be employed for any number of variables. But, it is generally used up to six variables. Beyond six variables, it becomes very complex.

4.1.2 Representation of SOP and POS on K-Map

The main objective of the K-Map is to simplify the Sum of Products and the Product of Sums expressions. Based on this, K- maps can be classified into two types.

- Minterm K – Map

- Maxterm K – Map

Minterm K-map is used to represent the Sum of Product expressions, whereas Maxterm K-map is used to represent the Product of Sum expressions.

Representation of Minterms on the K-Map

Figure 2 shows the truth table for some function with two variables, X, and Y. With two variables, we can have four minterms m0, m1, m2, and m3. Note that while writing minterms lower case 'm' is used.

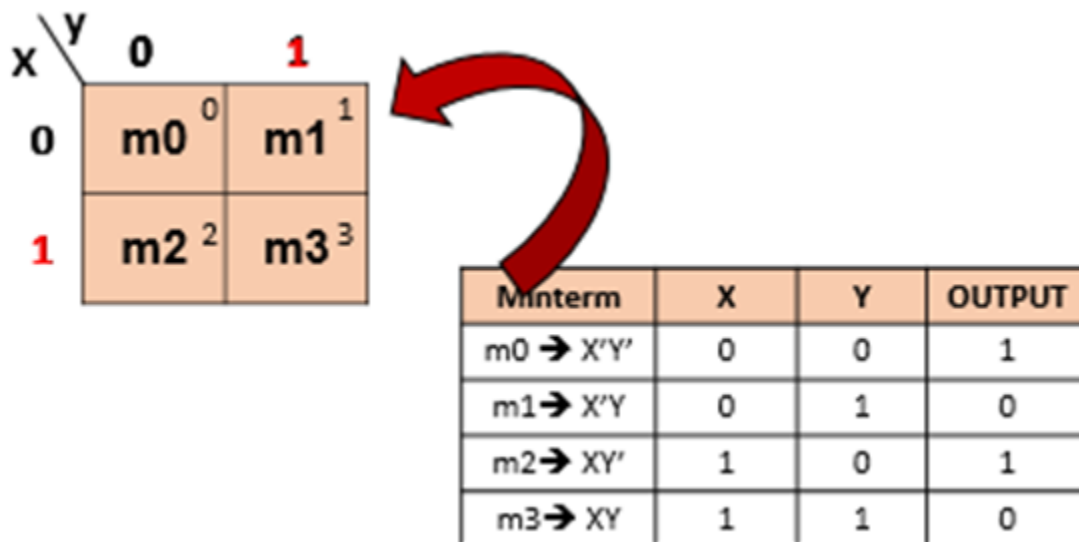


Figure 2: Minterm K – Map.

These four minterms are mapped to 2 variable K-map with four cells. These four cells are numbered as cell 0, 1, 2, and 3. To determine the position of minterms on this K-Map, write X and Y on the K-map. These two variables can take values 0 and 1. Note that variable X is MSB, and Y is LSB.

Minterms for different values of X and Y are given below:

- X = 0, Y = 0 , m0 □ X'Y'
- X = 0, Y = 1 , m1 □ X'Y
- X = 1, Y = 0 , m2 □ XY'
- X = 1, Y = 1 , m3 □ XY

Minterm m0 corresponds to x = 0 and y = 0 and is placed in the 0th cell. Minterm m1 corresponds to x = 0 and y = 1 and is placed in the 1st cell. The 0th cell and 1st cell are adjacent cells, and they differ in the value of one variable, i.e., for these two cells, X is 0, Y is 0 for the first cell, and Y is 1 for the second cell. Minterm m2 corresponds to X = 1 and Y = 0 and is placed in the 2nd cell. 0th and 2nd cells are adjacent cells, and they differ in the value of the X variable. The value of the Y variable for both these cells is 0. Minterm m3 corresponds to X = 1 and Y = 1 and is placed in the 3rd cell. 1st and 3rd cells are adjacent cells and 2nd and 3rd cells are adjacent cells.

In the case of a minterm K-map, logic '1' is placed in all those cells for which the output is '1', and '0' is placed in all those cells for which the output is '0'. For minterm m0, the output is 1. Enter 1 in the 0th cell. For minterm m1, the output is 0. Enter 0 in 1st cell. For minterm m2, the output is 1. Enter 1 in the 2nd cell. For minterm m3, the output is 0. Enter 0 in the 3rd cell. Figure 3 shows the mapping of 0's and 1's in the truth table to K-Map cells.

x \ y	0	1
0	1 ⁰	0 ¹
1	1 ²	0 ³

Minterm	X	Y	OUTPUT
m0 $\rightarrow X'Y'$	0	0	1
m1 $\rightarrow X'Y$	0	1	0
m2 $\rightarrow XY'$	1	0	1
m3 $\rightarrow XY$	1	1	0

Figure 3: Mapping of output to K-Map cells.

To simplify a Boolean expression, we have to systematically group these 1's in the K-map.

Representation of Max Terms on the K-Map

There are four Maxterms M0, M1, M2, and M3, corresponding to four input combinations. In max term representation, we write X when X = 0, and X' when X = 1. Minterms for different values of X and Y are given below:

- X = 0, Y = 0, M0 $\square X + Y$
- X = 0, Y = 1, M1 $\square X + Y'$
- X = 1, Y = 0, M2 $\square X' + Y$
- X = 1, Y = 1, M3 $\square X' + Y'$

x \ y	0	1
0	M0 ⁰	M1 ¹
1	M2 ²	M3 ³

Maxterm	X	Y	OUTPUT
M0 $\rightarrow X+Y$	0	0	1
M1 $\rightarrow X+Y'$	0	1	0
M2 $\rightarrow X'+Y$	1	0	1
M3 $\rightarrow X'+Y'$	1	1	0

Figure 4: Maxterm K – Map.

Figure 4 shows the Maxterm K-map. The placement of Maxterms in K-map is similar to that of minterm placement. In the case of a Maxterm K-map, '0' is placed in all those cells for which the output is '0'. '1' should be placed for input entries corresponding to a '1' output.

4.1.3 Procedure for Boolean Expression Simplification

Following is the general Procedure for Boolean Expression Minimization using K- Map.

1. Generate K – map.

To generate a K-map, figure out the number of variables in the given Boolean expression. The number of cells in the K-Map depends on the number of variables. For n variables, there will be 2^n cells.

For example: for a 2-variable Boolean expression, 4 cell K-Map is needed. For three variables, $2^3 = 8$ cells K-map is needed. If Minterm K-map is used, then enter 1 in K-map cells, only for the minterm whose output is 1. But if Maxterm K-map is used, then enter 0 only for the Maxterm whose output is 0.

2. Group all adjacent 1's (0s, in the case of Maxterm K-map).

The number of 1's in a group must be a power of 2. A group of 1 cell, 2 cells, 4 cells, etc. can be formed. But a group with 3 cells or 5 cells cannot be formed because they are not a power of two.

3. Write a Boolean expression.

Once groups are formed, the next step is to write the product (in case of Minterm K-map) / sum (in case of Maxterm K-map) terms. Identify the variables common in a group. Also, identify 1s (0s in case of Maxterms) that are not part of any groups and write expressions for the same which will contain all the variables.

4. Write SOP or POS terms obtained in step 3.

Two and Three-Variable K-Map

4.2.1 Two-Variable K-Map

Two variable K-map has four cells that can accommodate 4 Minterms or Maxterms. The position of Minterm or Maxterm depends on the value of two input variables. For Boolean expression simplification, adjacent cells are grouped together, and the resulting term will contain fewer literals or variables than the original term.

In a two-variable k-map, there are two ways to group the cells.

- Two adjacent cells and
- Four adjacent cells

Figure 1 shows the five cases of cell grouping. Two cells are said to be adjacent, if they differ in only one variable. In Figure 1(A) topmost two cells are adjacent since x is the same for both cells, but y is different that is, $Y = 0$ for cell 0 and is equal to 1 for cell 1. So, the common variable is X , and the Boolean expression for this grouping is X'

In Figure 1(B), the bottom two cells are adjacent. The Boolean expression for this grouping is X . In Figure 1(C), the leftmost two cells are adjacent. In this case, y is common and is equal to 0. Hence, the Boolean term is Y' . In Figure 1(D), for the rightmost cell, y is common and is equal to 1. Hence, the Boolean term is Y .

Two variable K- map also allows the grouping of four cells, which is shown in Figure 1(E). This results in final expression 1. This is because, for all the combinations of inputs, the output is one.

Problem 1: Minimize the following expression using two variable K-Map.

$$f(X, Y) =$$

$$\sum m(1,2,3)$$

$$\sum m(1,2,3)$$

Solution:

The above function has three Minterms, m_1 , m_2 , and m_3 , for which the output is 1. Fill the K-Map cells corresponding to these Minterms with 1. Figure 2 shows the 2- variable K-map grouping. The final Boolean expression is

$$f(X, Y) = X + Y$$

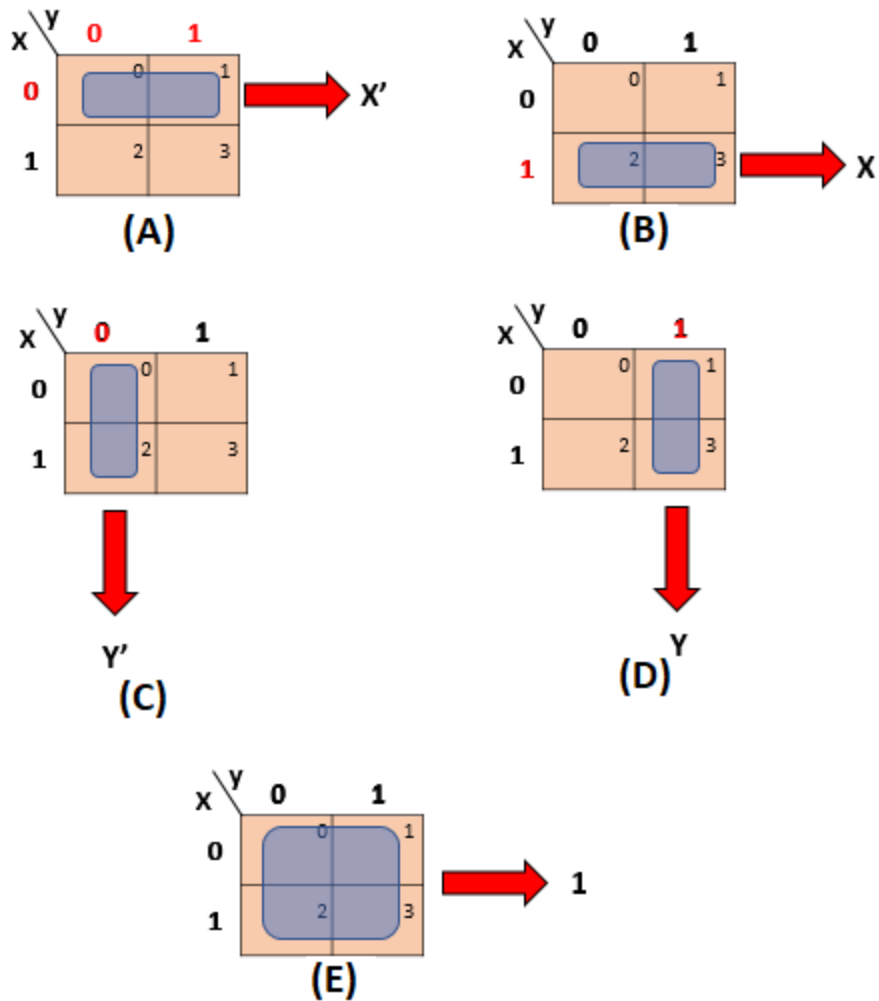


Figure 1: 5 Cases of 2 variable K-Map grouping.

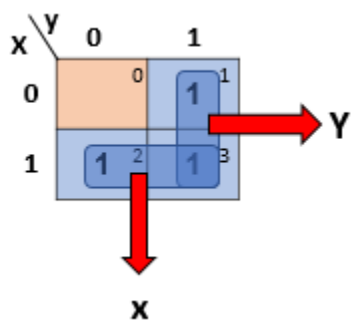


Figure 2: K-map grouping for problem 1.

Problem 2: For the given truth table, obtain a simplified Boolean expression using K-Map.

X	Y	OUTPUT
0	0	1
0	1	0
1	0	0
1	1	1

Solution:

First, write the minterms for the given truth table. The next step is identifying the minterms for which the output is 1. Output is 1 for minterms 0 and 3. m0 corresponds to cell 0, and m3 corresponds to cell 3. Place 1 in those cells. As seen in Figure 3, these two cells are diagonal, hence, cannot be grouped. So, group them individually. The final expression is $F(X,Y) X'Y' + XY$.

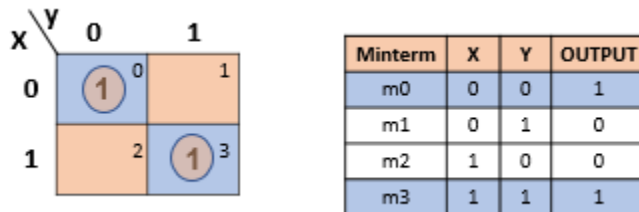


Figure 3: Two variable K-Map grouping for Problem 2.

4.2.2 Three Variable K-Map

Three variable K-Map has eight cells. These eight cells are organized in the form of 2 rows and four columns. Figure 4 shows the three-variable truth table and corresponding K- Map. Note that X is MSB, and Z is LSB. While numbering the cells, the cell adjacency rule needs to be followed. For example, YZ value for column 2 is not 10, but it is 11. Therefore, row 0 cells will have YZ values as 00, 01, 11, and 10.

As seen from the truth table, the output is 1 for Minterms m1, m3, m5, and m6. So fill up corresponding cells in K-Map with 1. Figure 9 shows grouping of cells.

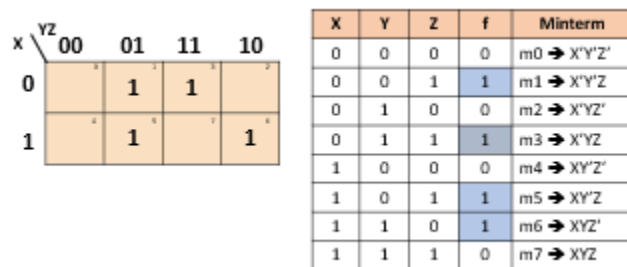


Figure 4: Three variable truth table and corresponding K-Map.

In general, a three-variable K-Map can have 1, 2, 4, and 8 cell grouping. 1-cell, 2-cell, and 3-cell grouping will result in 3, 2, and 1 variable product terms, respectively. Figure 5 shows example groupings.

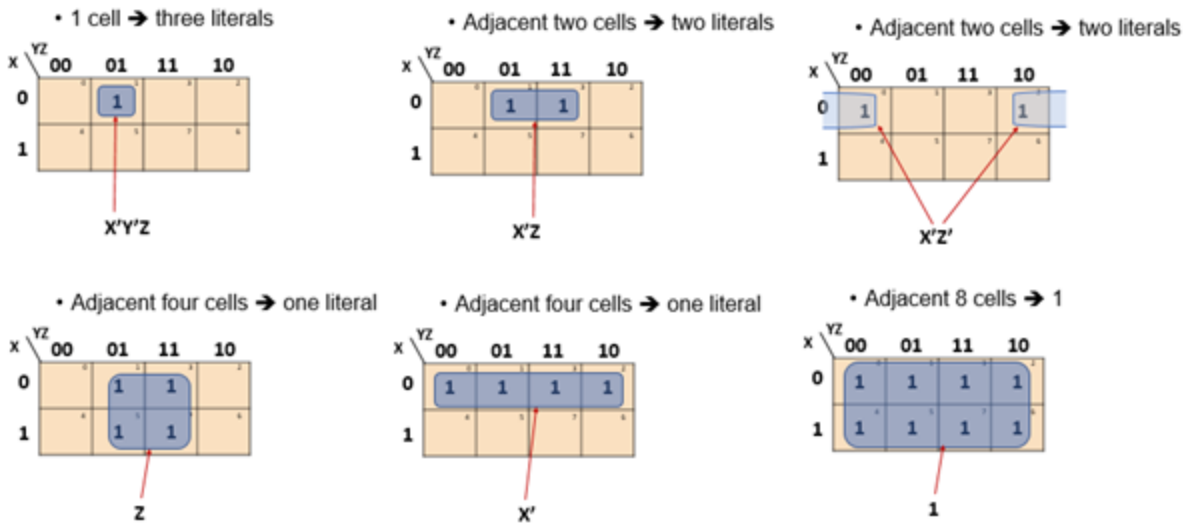


Figure 5: Example groupings – Three variable K-Map.

4.2.3 Three Variable K-Map: Example

Problem 1:

(a) Obtain the SOP expression for the given Truth table (Refer Figure 6)

(b) Obtain the SIMPLIFIED SOP expression for the given Truth table using K-Map

X	Y	Z	f	Minterm
0	0	0	0	m0 $\Rightarrow$ X'Y'Z'
0	0	1	1	m1 $\Rightarrow$ X'Y'Z
0	1	0	0	m2 $\Rightarrow$ X'YZ'
0	1	1	1	m3 $\Rightarrow$ X'YZ
1	0	0	0	m4 $\Rightarrow$ XY'Z'
1	0	1	1	m5 $\Rightarrow$ XY'Z
1	1	0	1	m6 $\Rightarrow$ XYZ'
1	1	1	1	m7 $\Rightarrow$ XYZ

Figure 6: Three variable truth table.

Solution (a) :

f =

$$\sum(m1, m3, m5, m6, m7)$$

$$\sum(m1, m3, m5, m6, m7)$$

=

$$\sum(1, 3, 5, 6, 7)$$

$$\sum(1, 3, 5, 6, 7)$$

$$= X'Y'Z + X'YZ + XY'Z + XYZ' + XYZ$$

Solution (b): For Minterms m1, m3, m5, m6, and m7 the output is 1. Cells corresponding to these minterms are entered with 1. Grouping of four 1's yields a term Z, whereas grouping of two 1's yields a term XY. Refer to Figure 7. The simplified Boolean expression is

$$f(XYZ) = Z + XY$$

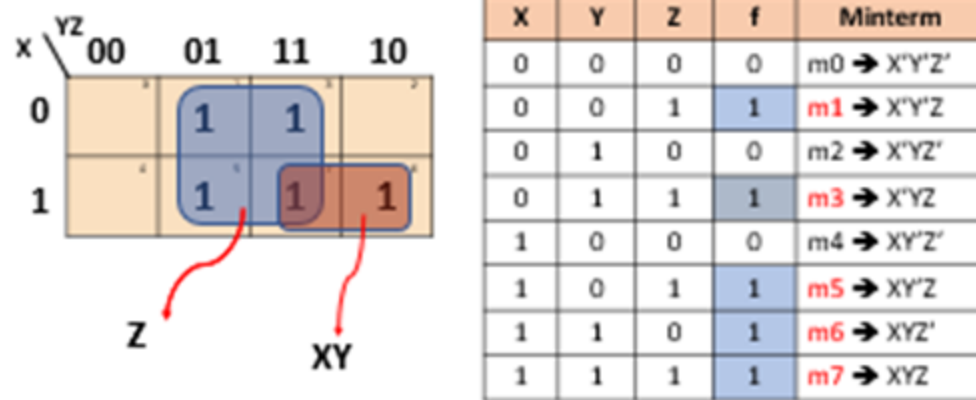


Figure 7: 3 variable K – Map and grouping.

Four-Variable K-Map

4.2.1 Four-Variable K-Map

Consider a truth table with four variables W, X, Y, and Z, where W represents the most significant bit variable, and Z represents the least significant variable. With four input variables, 16 different combinations representing 16 minterms m0 through m15 is obtained. To accommodate these 16 minterms, 16 cell K- Map is used. Figure 1 shows the placement of minterms in K-Map.

Minterms m0 to m2 are placed in Row 0. Note the position of m2 and m3. Minterm m2 is placed in cell 3, whereas m3 is placed in cell 2. In row1, minterms m4, m5, m6 and m7 is placed. Minterms m8, m9, m10 and m11 is placed Row3 whereas remaining minterms m12, m13, m14 and m15 are placed in row2.

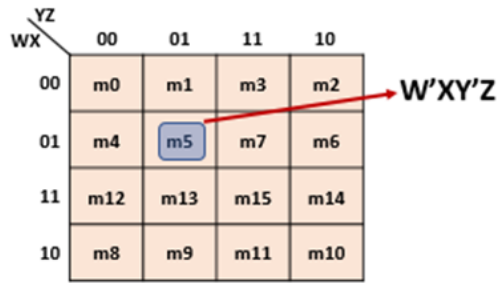
YZ WX	00	01	11	10
00	m0	m1	m3	m2
01	m4	m5	m7	m6
11	m12	m13	m15	m14
10	m8	m9	m11	m10

W	X	Y	Z	MINTERM
0	0	0	0	m0
0	0	0	1	m1
0	0	1	0	m2
0	0	1	1	m3
0	1	0	0	m4
0	1	0	1	m5
0	1	1	0	m6
0	1	1	1	m7
1	0	0	0	m8
1	0	0	1	m9
1	0	1	0	m10
1	0	1	1	m11
1	1	0	0	m12
1	1	0	1	m13
1	1	1	0	m14
1	1	1	1	m15

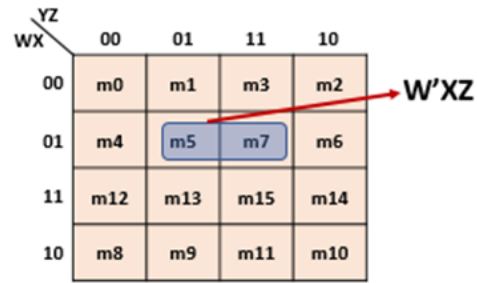
Figure 1: 4 variable truth table and K-Map.

To simplify a Boolean expression, group the minterms. In 4 variable K- map, 1-cell, 2-cell, 4-cell, 8-cell, and 16-cell grouping is possible. Single-cell grouping results in 4 variable terms. The expression for m5 is $W'XY'Z$. Two-cell grouping results in 3 variable term. For example, for this particular grouping, the expression is $W'XZ$. Four cells grouping results in 2 variable term. For the current example, the simplified term is XZ . Figure 2 shows examples of 4- variable K-map grouping.

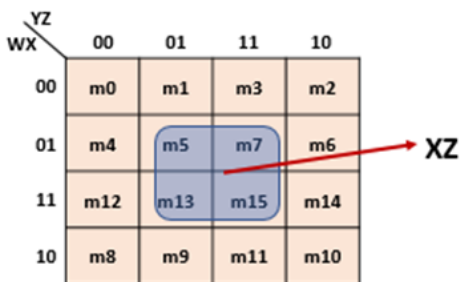
- Single cell grouping results in 4 variable term



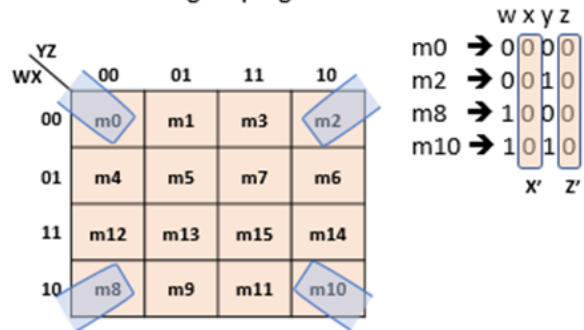
- Two cells grouping results in 3 variable term



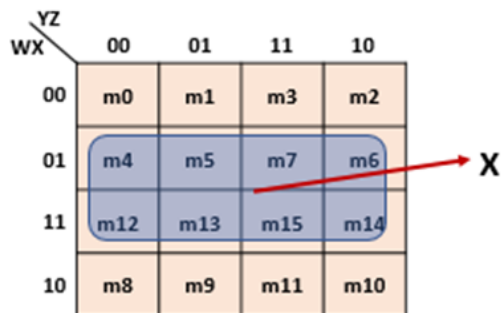
- Four cells grouping results in 2 variable term



- Four cells grouping results in 2 variable term



- 8 cell grouping results in 1 variable term



- 16 cell grouping results in 1

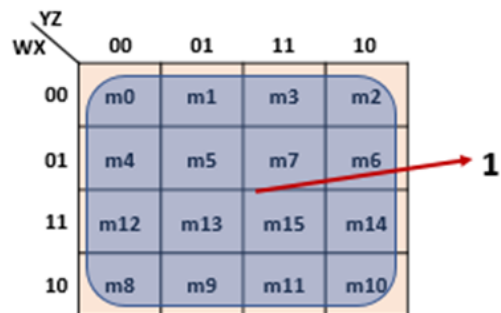


Figure 2: Sample cell grouping in 4 variable K-Map.

4.2.2 Four Variable K-Map: Examples

Example 1: Simplify the following expression using K-Map for the four variable WXYZ.

$$F = m1 + m3 + m5 + m7 + m8 + m9 + m10 + m12 + m13$$

Solution: Minterm placement and cell grouping is shown in Figure 3. The simplified expression is given by

$$F = W'Z + WY' + WX'Z'$$

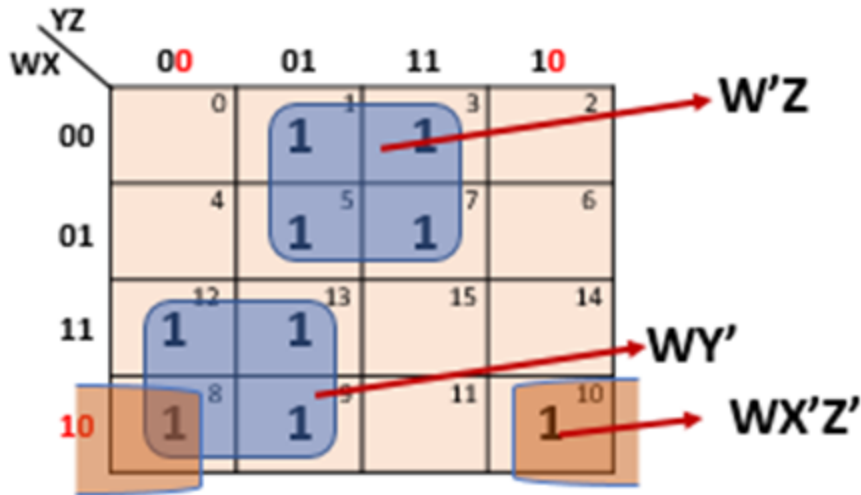


Figure 3: Four variable K-Map for example 1.

Example 2: Obtain the simplified logical expression for

$$F = WXYZ + W'X'Y'Z' + WX'Y + WX$$

Solution: F is a function of four variables $WXYZ$. So, 16 cell K-Map is needed. In the given expression, there are four terms. These four terms correspond to four groups in the K map. Locate these four groups in the K –Map. 1 cell grouping gives 4 variable term, 2 cell grouping gives 3 variable term, whereas 4 cell grouping gives 2 variable term.

- The first term has four variables $WXYZ$. Hence it corresponds to 1 cell grouping. Since all the variables are in un-complemented form, their value is equal to 1111. Locate a cell that corresponds to 1111 on the K – map. Write 1 in that cell. Refer to Figure 4 (a).
- The second term also has four variables in complement form. So $WXYZ$ value will be 00 00. Since there are four variables in the term, the term maps to 1 cell grouping. Locate a cell that corresponds to $WXYZ = 00\ 00$ and write 1 in that cell. Refer to Figure 4 (b).
- The third term is $WX'Y$. Since there are three variables in the term, the term maps to 2 cell grouping. The missing variable in the term is Z . Search for two cells where $W = 1$, $X = 0$, $Y = 1$, and $Z = 0$ or 1 . Write 1 in these two cells. Refer to Figure 4 (c).
- The last term contains 2 variables and maps to 4 cell grouping. Write 1 in these 4 cells. A group of 4 cells gives WY . Four-cell grouping gives WX term. The last one is a one-cell grouping which gives us $W'X'Y'Z'$. Refer to Figure 4 (d).

As seen from the K-Map, the given expression is not a simplified expression. Regrouping of terms is needed and is shown in Figure 5. Obtain terms for each grouping. Adding all the terms gives the simplified function $f = W'Z + WY' + WX'Z'$

$$F = WXYZ + W'X'Y'Z' + WX'Y + WX$$

1 Cell

YZ \ WX	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

(a)

$$F = WXYZ + W'X'Y'Z' + WX'Y + WX$$

1 Cell

YZ \ WX	00	01	11	10
00	1	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

(b)

$$F = WXYZ + W'X'Y'Z' + WX'Y + WX$$

2 Cell

YZ \ WX	00	01	11	10
00	1	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

(c)

$$F = WXYZ + W'X'Y'Z' + WX'Y + WX$$

4 Cell

YZ \ WX	00	01	11	10
00	1	1	3	2
01	4	5	7	6
11	1	1	1	1
10	8	9	1	1

(d)

Figure 4: Example 2 – Each term mapping.

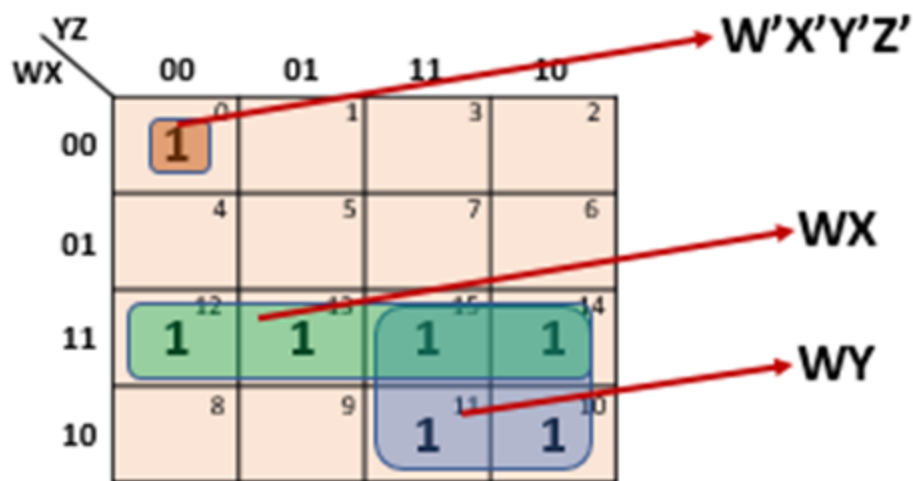


Figure 5: Example 2: Regrouping of 1's.

Example 3: Obtain minimal sum of product expressions for the function given below

$F(W,X,Y,Z) =$

$\sum(0,1,2,5,8,9,10)$

$\sum(0,1,2,5,8,9,10)$

Solution: Figure 6 shows the k-map for the given Boolean expression with four variables. The simplified expression is given by

$F(W,X,Y,Z) = X'Z' + W'Y'Z + X'Y'$

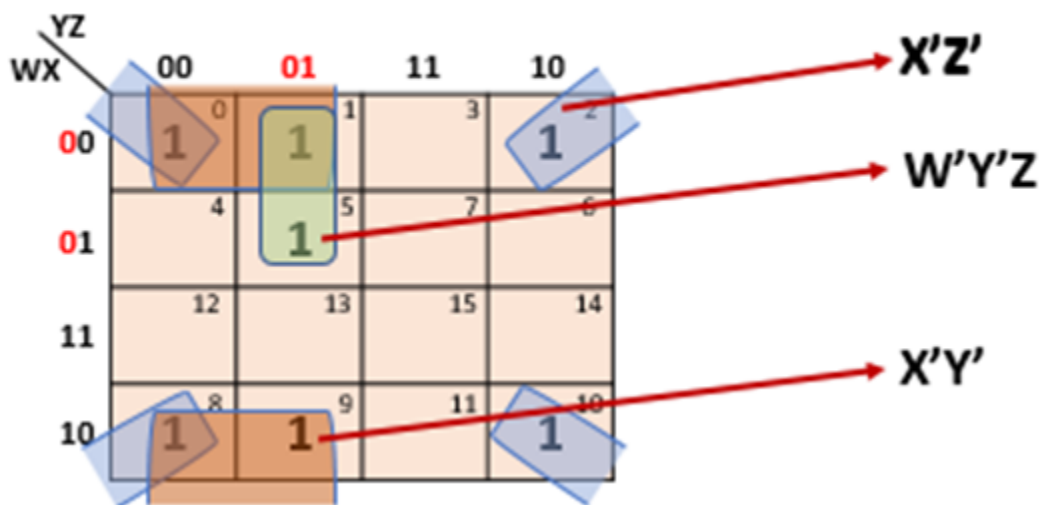


Figure 6: K-map and grouping for example 3.

4.2.3 Don't Care Condition

Usually while simplifying Boolean expressions using K-Map, input, and output conditions are specified completely. For example: $F(X, Y, Z)$ equal $m_0 + m_3 + m_5$. This function yields a 1 output for minterms m_0 , m_3 , and m_5 . For other minterms, the output is 0. For many of the practical applications, it is not necessary to specify input and corresponding output conditions completely. An example is BCD numbers. BCD numbers are encoded using four bits. With four bits, we can have 16

binary combinations. Out of these 16 combinations, only ten combinations are useful. The remaining six combinations are not used as they are not valid BCD codes. Those combinations can yield 0 or 1 as output. Such output conditions are called “Don’t care conditions”. In two cases, don’t care conditions can occur in a Boolean function. First, a set of inputs never occurs. Second, a set of inputs is not a valid combination for a digital circuit. Such input combinations representing Don’t care are often used to form a larger group of cells. A Don’t Care cell in a K-Map is represented by a cross. The procedure to be adopted while solving a Boolean Expression with a Don’t care condition is as follows:

Step 1: fill the cells with 1 as per SOP terms specified in the function

Step 2: Fill the cells with X as per the Don’t care specified in the function

Step 3: Group the cells with adjacent 1’s and X’s

Step 4: Write Minterm for each group and sum it to form an SOP expression

Example 1: Minimize the SOP expression using K-Map.

$$F(W, X, Y, Z) = \sum m(1, 5, 6, 12, 13, 15) + d(2, 4)$$

Solution:

F is a function with four variables W, X, Y, and Z. First part of the function is minterms for which the output is 1. The second part is don’t care. The K-Map is of 16 cells. Fill up this K-map with minterms. The next step is to add don’t care in the K-map. There are two cells corresponding to two don’t cares. Enter X in those cells. Enter X in the cell corresponding to minterms m2 and m4. The Next step is grouping. Wherever it is possible, while grouping, include the cell which contains X. The last step is to add all the terms obtained after grouping the cells. Figure 7 shows the K-Map with the “don’t care condition.” The final expression is

$$F(W, X, Y, Z) = XY' + W'Y'Z + WXZ + W'YZ'$$

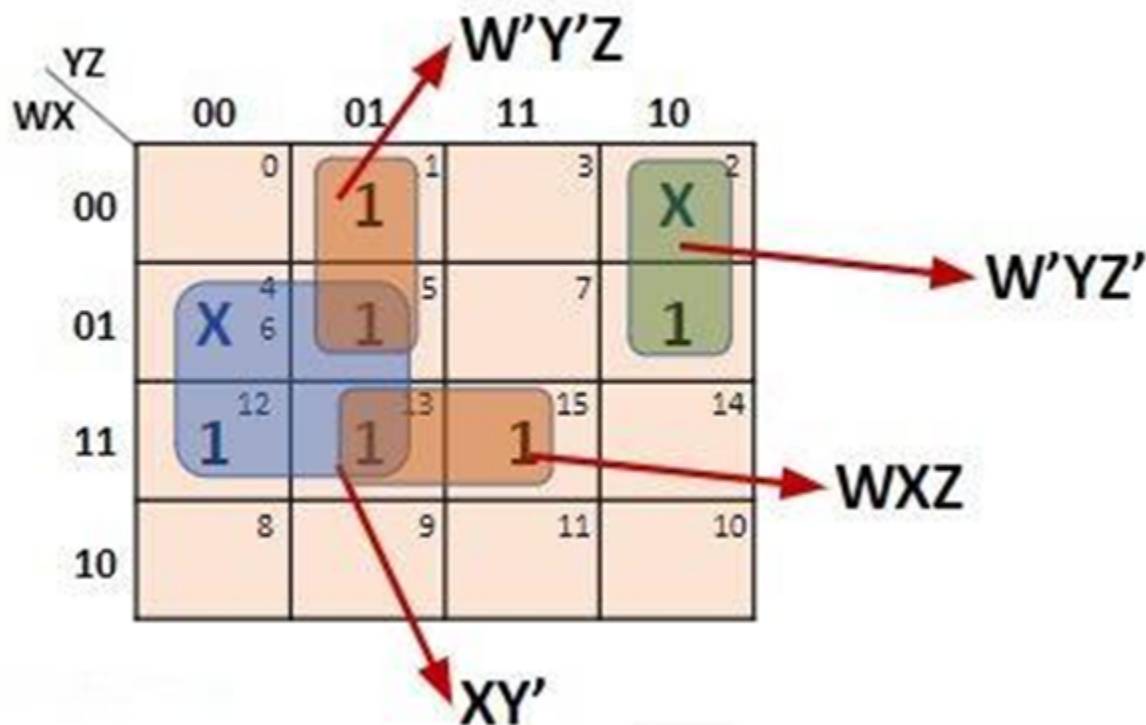


Figure 7: K-Map simplification with don’t care condition.

POS Expression Simplification

4.4.1 POS Expression Simplification using Three-Variable K-Map

The simplification of POS expression using K-Maps is simple and is the same as that of SOP expression simplification, except that with POS expressions, 0s representing the standard sum terms are placed on the Karnaugh map instead of 1s. Consider a truth table with three variables X, Y, and Z. X is MSB, and Z is LSB. M0 through M7 are the eight Maxterms corresponding to 8 combinations of the input. K-Map contains eight cells. The first four Maxterms, M0, M1, M2, and M3, are mapped to row 0 where X equals 0. The next four Maxterms, M4, M5, M6, and M7, are mapped to row 1 where X equals 1. Sum terms for various X, Y, and Z values are shown in Figure 1.

		YZ			
		00	01	11	10
X					
0		M0	M1	M3	M2
1		M4	M5	M7	M6

		YZ			
		00	01	11	10
X					
0		$X + Y + Z$	$X + Y + Z'$	$X + Y' + Z'$	$X + Y' + Z$
1		$X' + Y + Z$	$X' + Y + Z'$	$X' + Y' + Z'$	$X' + Y' + Z$

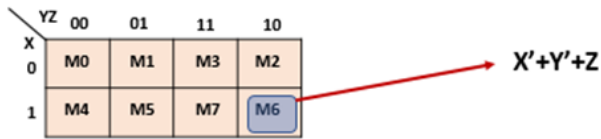
Truth Table:			
X	Y	Z	MAXTERM
0	0	0	M0
0	0	1	M1
0	1	0	M2
0	1	1	M3
1	0	0	M4
1	0	1	M5
1	1	0	M6
1	1	1	M7

Figure 1: Three variable K-Map representing Maxterms.

Note that when a variable is 0, it is written in the un-complemented form and when it is 1, it is written in the complemented form. For example, when XYZ is 000, the sum term is $X + Y + Z$. All three variables are expressed in un-complemented form.

As far as the grouping is concerned, 1-cell, 2-cell, 4-cell, and 8-cell grouping is allowed. Note that the group should contain the power of two number of cells. For example, having three cell grouping is not possible since three is not a power of two. The group should be rectangular. Diagonal grouping is not allowed. 0 in the cell corresponds to a Maxterm. No 1s are allowed to be grouped. Every 0 in the K-map must be in at least one group. Overlapping and wrapping around are allowed. Figure 2 shows examples of grouping and the corresponding Maxterm expressions.

- Single cell grouping results in 3 variable term



- Two cell grouping results in 2 variable term



- Four cell grouping results in 1 variable term



- 8 cell grouping results in 0



Figure 2: Sample groupings and corresponding Maxterms.

1-cell grouping results in 3 variable sum term. For example, grouping a single cell containing Maxterm M6 results in $X' + Y' + Z$. 2-cell grouping results in 2 variable term. As you can see, the max terms M7 and M6 are grouped together. For this group, X and Y are common, and their values are 1 and 1, respectively. Hence the sum term is equal to $X' + Y'$. Four-cell grouping results in 1 variable term. Here also, we need to look at the common variable. Group has one common variable, that is, Z which is equal to 1. Hence the sum term is Z' . Eight-cell grouping results in 0.

Problem 1: Obtain simplified POS expression for the given truth table using K- Map as shown in Figure 3.

X	Y	Z	f	Minterm
0	0	0	0	$M0 \rightarrow XYZ$
0	0	1	1	$M1 \rightarrow XYZ'$
0	1	0	0	$M2 \rightarrow XY'Z$
0	1	1	1	$M3 \rightarrow XY'Z'$
1	0	0	0	$M4 \rightarrow X'YZ$
1	0	1	1	$M5 \rightarrow X'YZ'$
1	1	0	1	$M6 \rightarrow X'Y'Z$
1	1	1	1	$M7 \rightarrow X'Y'Z'$

Figure 3: Truth Table for problem 1.

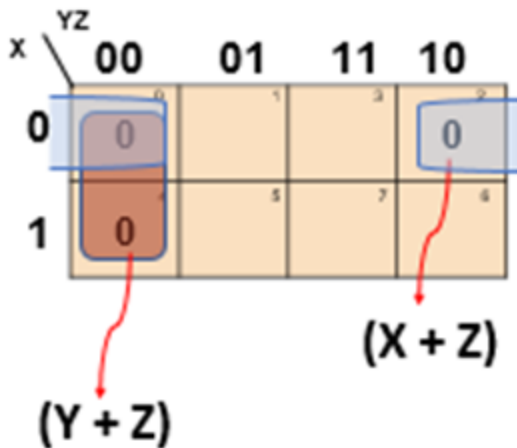


Figure 4: K-Map for Problem 1.

The first step is to identify Maxterms for which the output function f equals 0. There are three places where f equals 0. that is, M0, M2, and M4. Insert zero in the K-Map corresponding to M0, M2, and M4. The next step is to group cells. Figure 4 shows the cell groupings and corresponding Maxterms. The final simplified POS expression is

$$F = (Y + Z) (X + Z)$$

4.4.2 POS Expression Simplification using Four-Variable K-Map

Four input variables provide 16 Maxterms M0 through M15. To accommodate these 16 minterms, a 16-cell K- Map is needed. Figure 5 shows the variable names, their values, and Maxterms on the K-Map. Maxterms M0 to M3 are placed in Row 0. Maxterms M4 to M7 are placed in Row 1. Maxterms M8 to M11 are placed in Row 3, and Maxterms M12 to M15 are placed in Row 2. To simplify a Boolean expression, Maxterms are grouped. In 4 variable K- Map, 1-cell, 2-cell, 4-cell, 8-cell, and 16-cell grouping is possible. Sample cell grouping is shown in Figure 5.

YZ WX				
	00	01	11	10
00	M0	M1	M3	M2
01	M4	M5	M7	M6
11	M12	M13	M15	M14
10	M8	M9	M11	M10

W	X	Y	Z	MINTERM
0	0	0	0	M0
0	0	0	1	M1
0	0	1	0	M2
0	0	1	1	M3
0	1	0	0	M4
0	1	0	1	M5
0	1	1	0	M6
0	1	1	1	M7
1	0	0	0	M8
1	0	0	1	M9
1	0	1	0	M10
1	0	1	1	M11
1	1	0	0	M12
1	1	0	1	M13
1	1	1	0	M14
1	1	1	1	M15

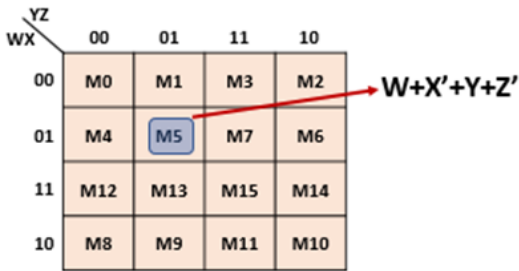
Figure 5: Four variable K-Map for POS simplification.

Single-cell grouping results in 4 variable term. Two-cell grouping results in 3 variable term. Four cells grouping results in 2 variable term. The last type of grouping involves all 16 cells in the K- Map.

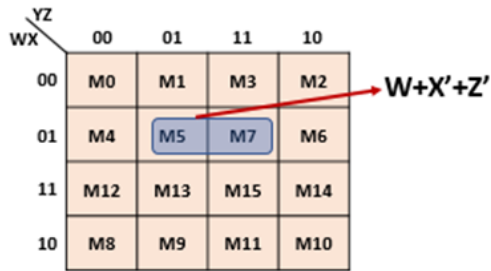
Problem 2: Simplify the POS Boolean function, which is given in the table.

Figure 6 shows the truth table and corresponding K-Map. For Maxterm representation, look for zeros in the output column. There are six places where the output is zero. Write 0 in those Maxterm positions. The next step is grouping which is shown in Figure 7.

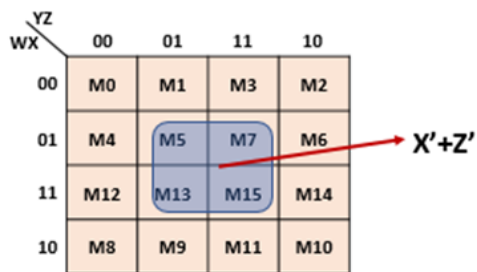
- Single cell grouping results in 4 variable term



- Two cells grouping results in 3 variable term



- Four cells grouping results in 2 variable term



- 16 cell grouping results in 0

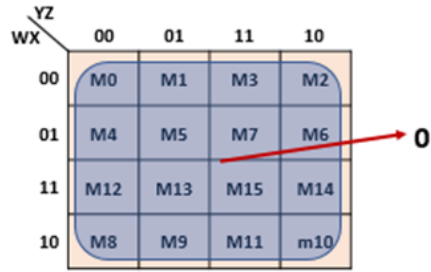
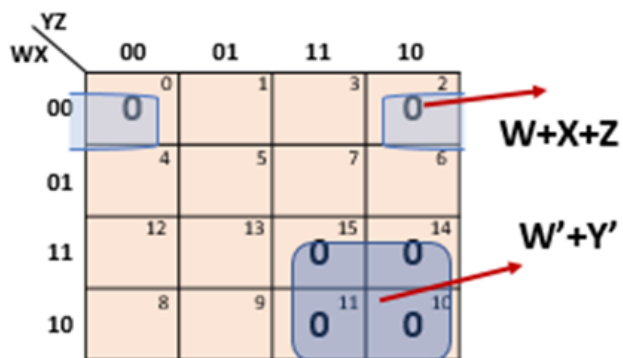


Figure 6: Sample cell groupings.



W	X	Y	Z	MAX term	F
0	0	0	0	M0	0
0	0	0	1	M1	1
0	0	1	0	M2	0
0	0	1	1	M3	0
0	1	0	0	M4	1
0	1	0	1	M5	1
0	1	1	0	M6	1
0	1	1	1	M7	1
1	0	0	0	M8	1
1	0	0	1	M9	1
1	0	1	0	M10	0
1	0	1	1	M11	0
1	1	0	0	M12	1
1	1	0	1	M13	1
1	1	1	0	M14	0
1	1	1	1	M15	0

Figure 7: Truth table and K-Map for Problem 2.

As you can see, there are two groups<click>. The first group contains four 1s and results in the term $W' + Y'$. The next group contains two 1s and results in three variable term $W + X + Z$.

The final POS expression is
 $F = (W' + Y') (W + X + Z).$